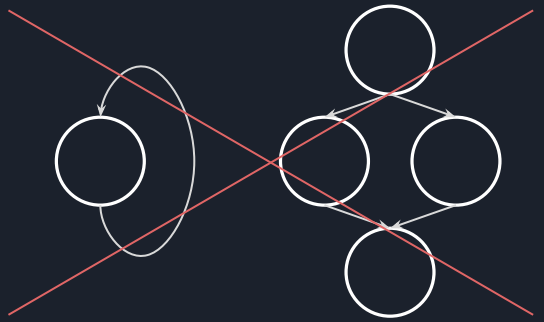
22. Tree

2025資訊研究社 算法班
Made with ❤️ by jheanlee

什麼是tree



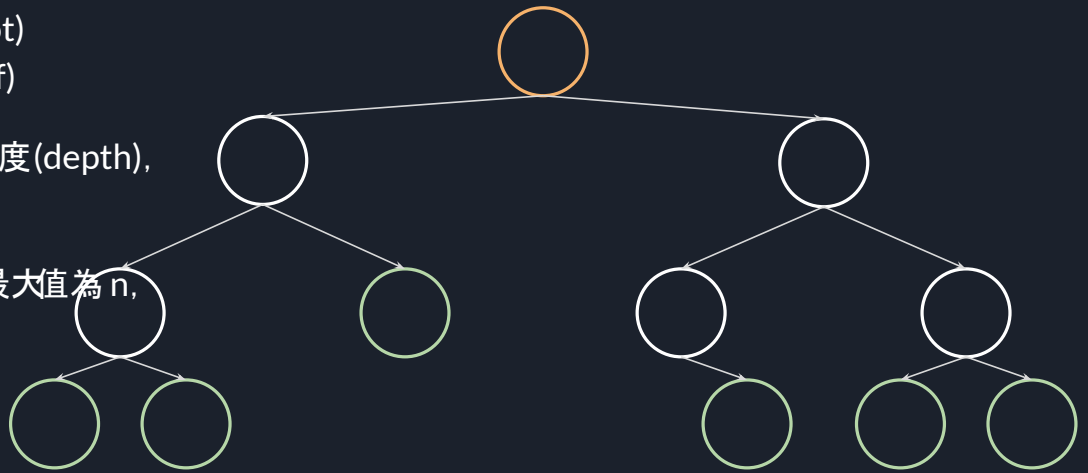
Tree 是一種有階級的抽象資料結構，其中**沒有環狀結構 (cycle) 存在**

每個節點(node) 都有零個或多個子節點(children)
若節點A有子節點B，則節點A為B之父節點(parent)

沒有父節點的節點稱為根節點(root)
沒有子節點的節點稱為葉節點(leaf)

從根節點到節點N的路徑長稱為深度(depth)，
根節點深度為0

若整個樹中所有節點的子節點數最大值為 n，
這棵樹為 n元樹



Tree的種類

- 順序性質
  - 無序樹/自由樹: 父子節點之間沒有順序關係
  - 有序樹/搜尋樹: 父子節點有一定的順序大小關係。(所有節點照一定的順序排列)
- 節點分布
  - 等差樹
    - 二元樹
      - 完美二元樹 (Perfect Binary Tree)
      - 完全二元樹 (Complete Binary Tree)
      - 平衡二元樹 (AVL Tree)
      - 二元搜尋樹 (BST)
    - 多元樹
  - 不等差樹

Tree的用途

作業系統: 以層狀結構儲存資料(使用 symlink 會產生環狀圖)

資料庫: 資料庫常使用搜尋樹增加取值的速度 (常使用B Tree 、B+ Tree 等)

自動完成: 以 trie (prefix tree) 推出可能的字詞

C++ std函式庫: set, map, heap (priority_queue) 都是使用 tree 實作

網域查找: 常使用反向域名 (reverse domain name notation) 表示，再用樹查找

實作 tree

```cpp
class Tree {
public:
  int val;
  vector<Tree *> children;

  Tree(int _val) {
    val = _val;
    children = vector<Tree *>();
  }
};
```

普通的樹

```cpp
class BinaryTree {
public:
  int val;
  BinaryTree *left;
  BinaryTree *right;

  BinaryTree(int _val) {
    val = _val;
    left = nullptr;
    right = nullptr;
  }
};
```

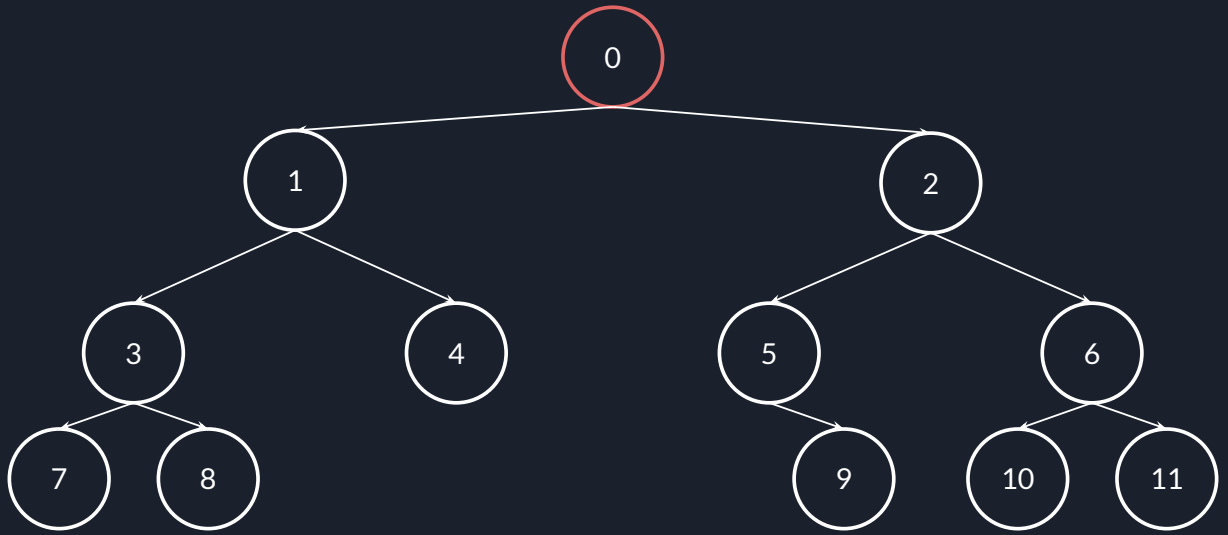
二元樹

Depth-First Search

深度優先搜尋法(depth-first search, DFS) 是用於遍歷 (traverse) 或搜尋 (search) 的演算法

- 從根出發，遇到分岔則會將其中一個分枝**遍歷到最深處**才遍歷其他分枝
  (相對於廣度優先搜尋法BFS)
- 將所在節點的所有子節點放入一個堆疊(stack，LIFO 後入先出)
  遞迴作法使用系統的記憶體堆疊(memory stack)

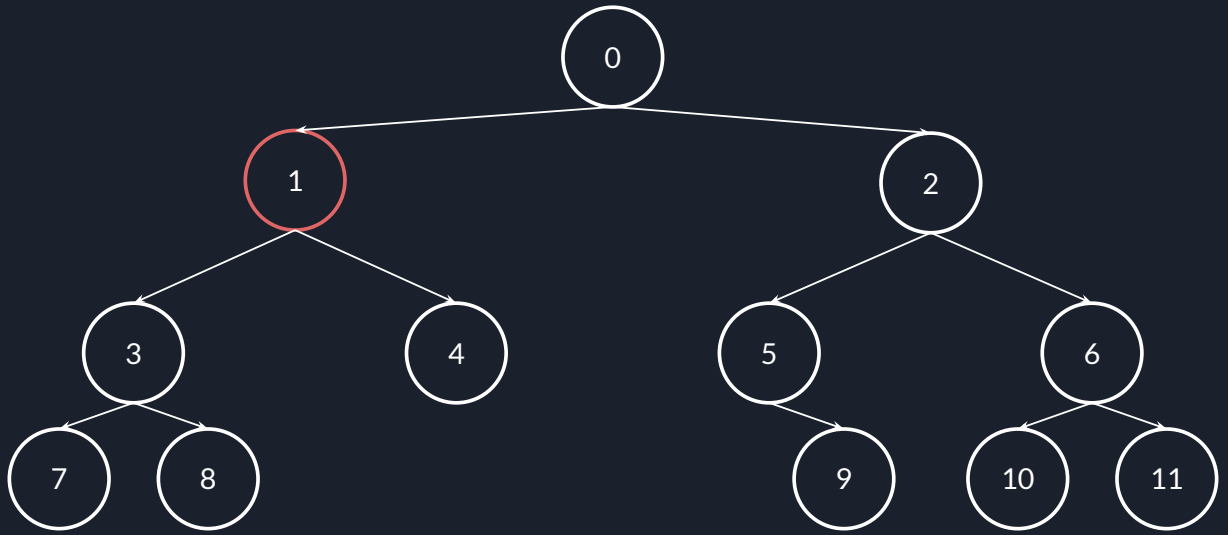
Depth-First Search



Visited: 0

Stack: [2, 1]

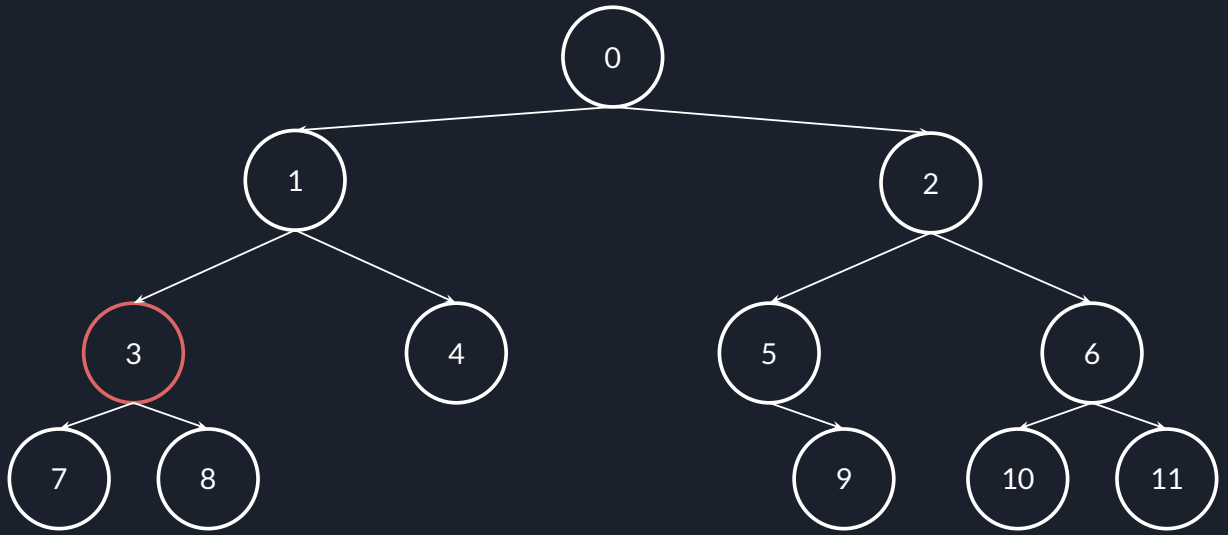
Depth-First Search



Visited: 0, 1

Stack: [2, 4, 3]

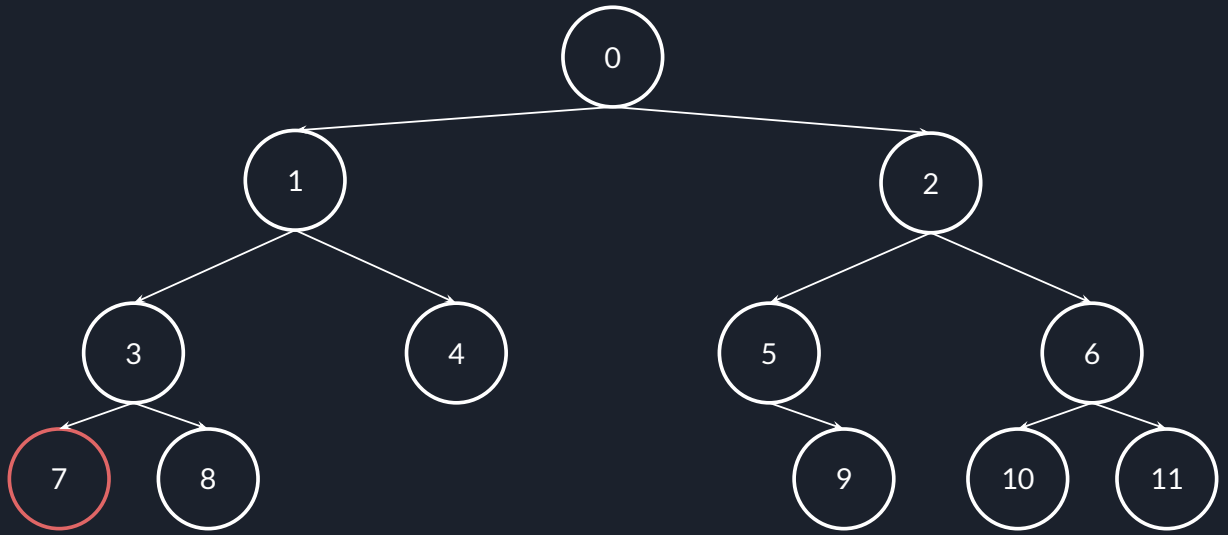
Depth-First Search



Visited: 0, 1, 3

Stack: [2, 4, 8, 7]

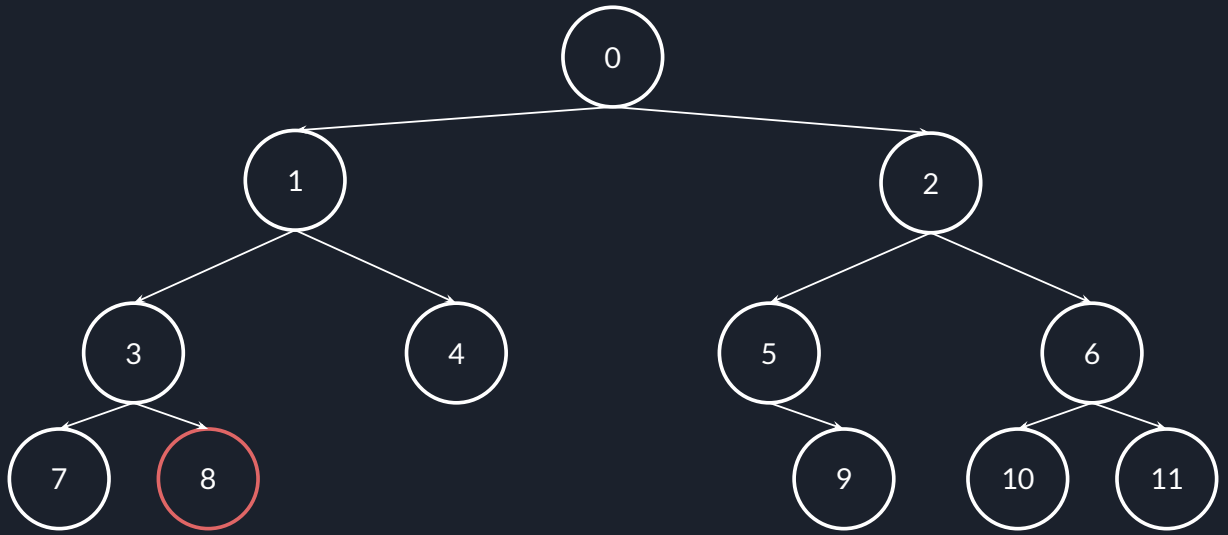
Depth-First Search



Visited: 0, 1, 3, 7

Stack: [2, 4, 8]

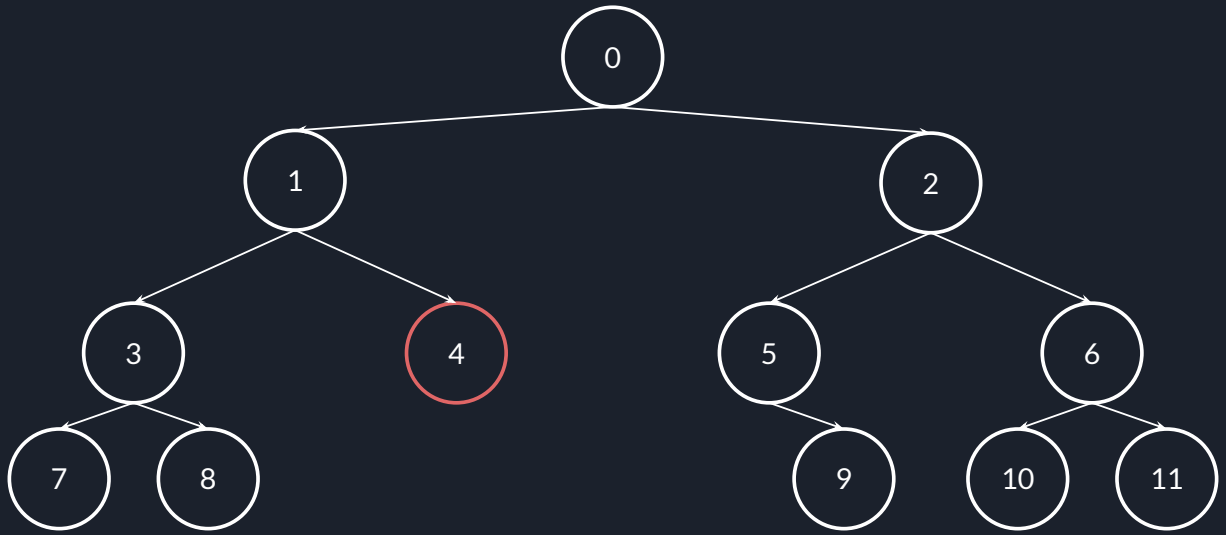
Depth-First Search



Visited: 0, 1, 3, 7, 8

Stack: [2, 4]

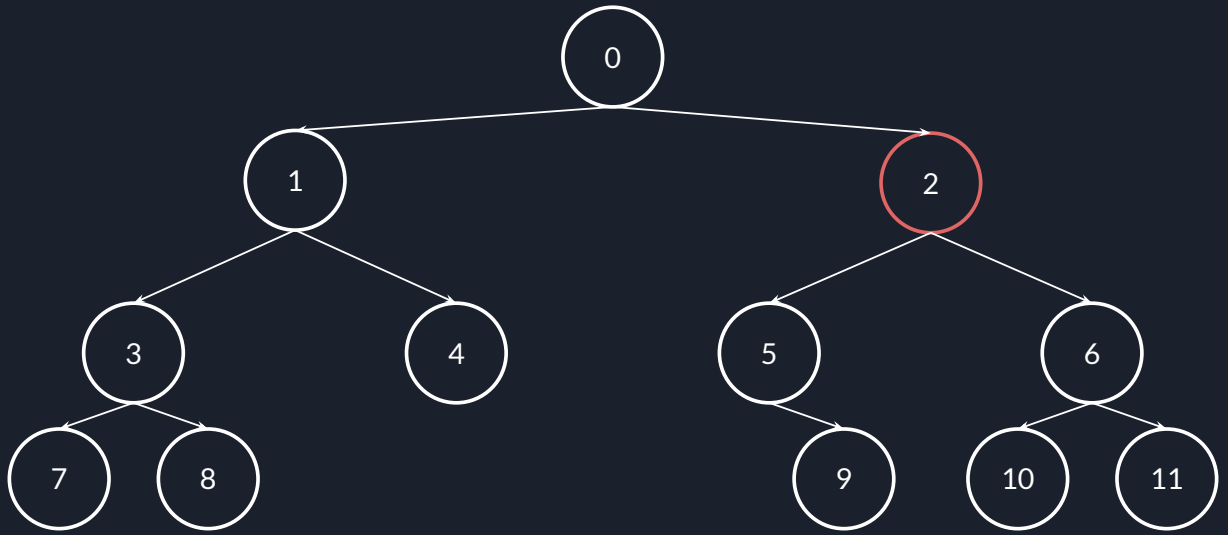
Depth-First Search



Visited: 0, 1, 3, 7, 8, 4

Stack: [2]

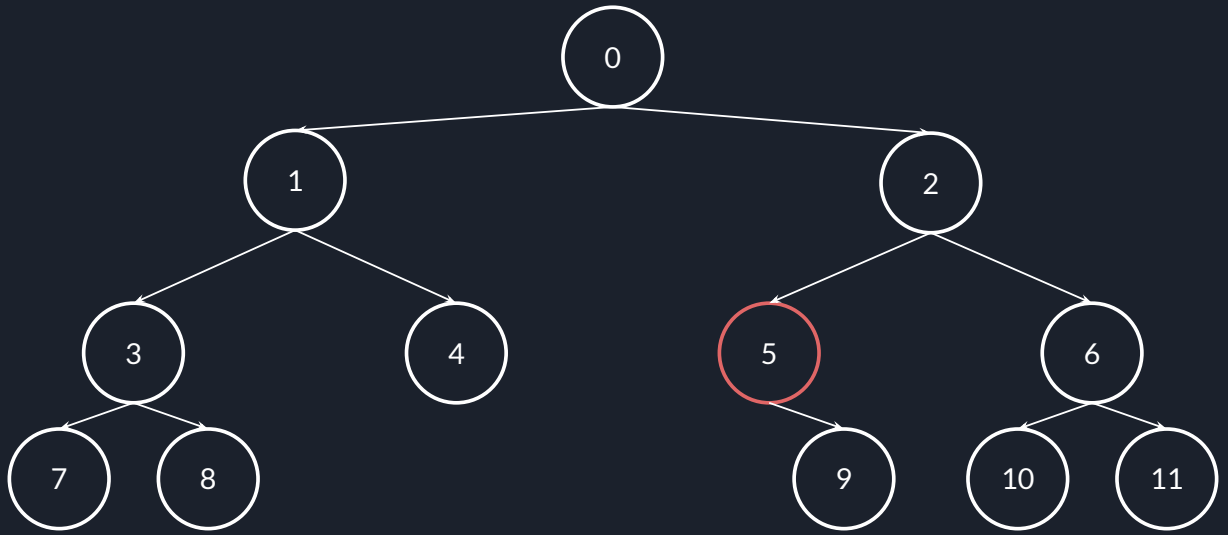
Depth-First Search



Visited: 0, 1, 3, 7, 8, 4, 2

Stack: [6, 5]

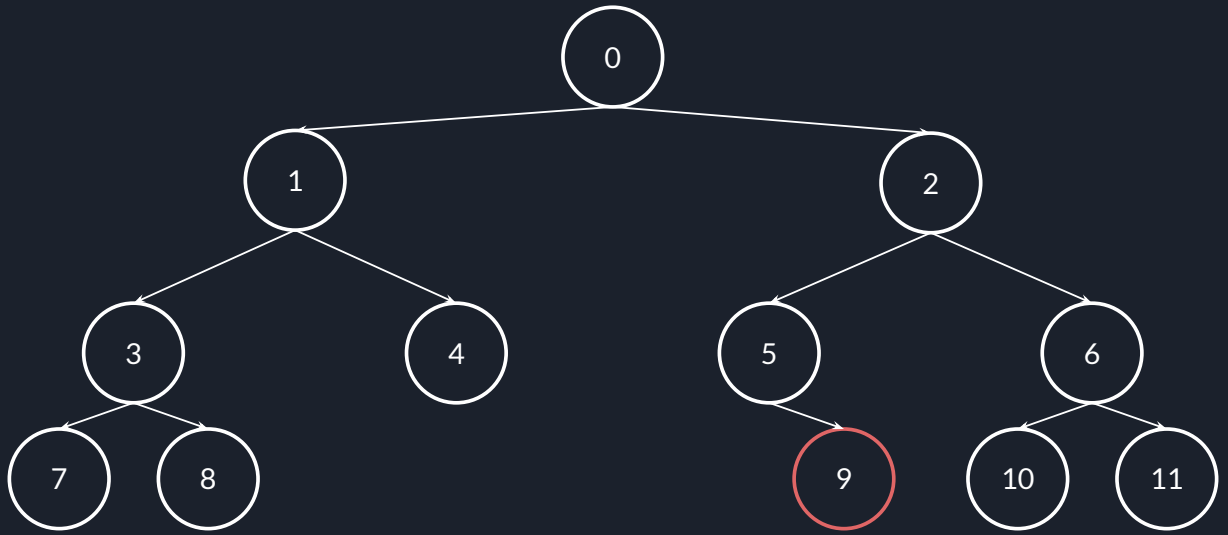
Depth-First Search



Visited: 0, 1, 3, 7, 8, 4, 2, 5

Stack: [6, 9]

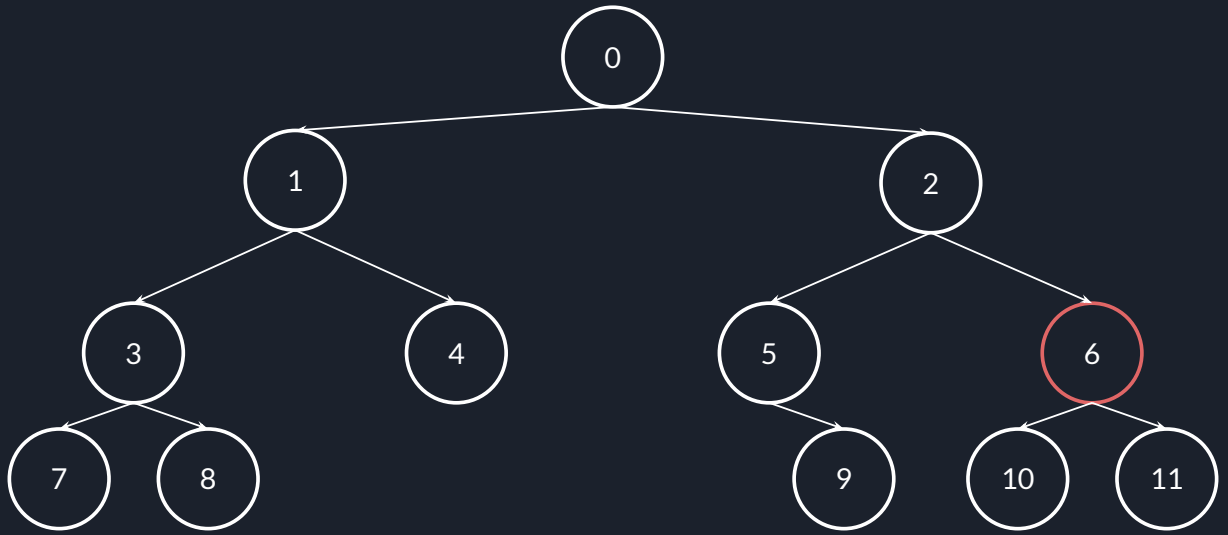
Depth-First Search



Visited: 0, 1, 3, 7, 8, 4, 2, 5, 9

Stack: [6]

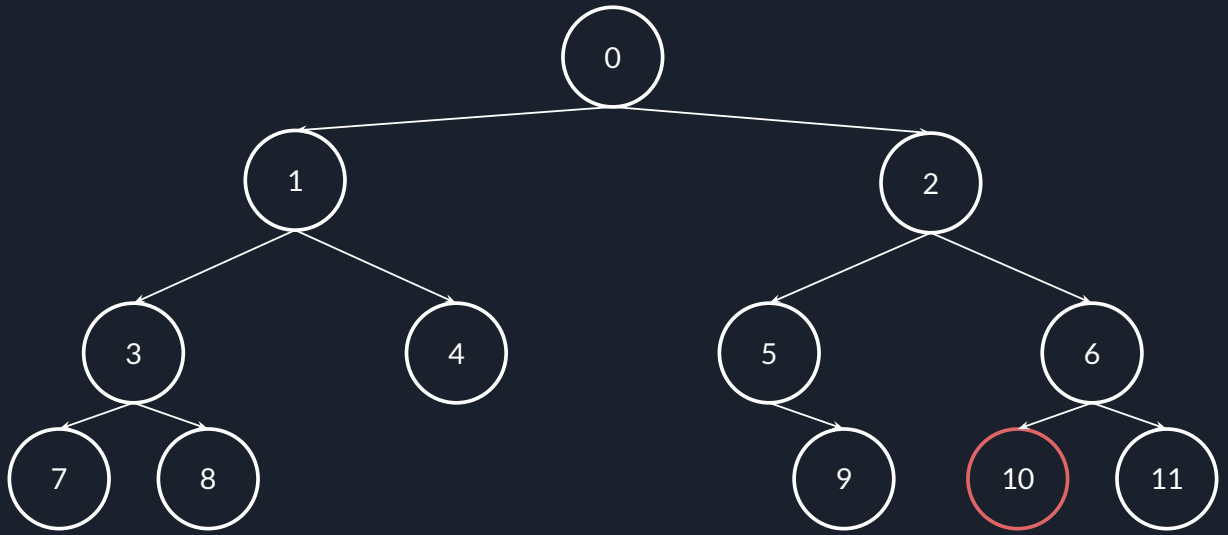
Depth-First Search



Visited: 0, 1, 3, 7, 8, 4, 2, 5, 9, 6

Stack: [11, 10]

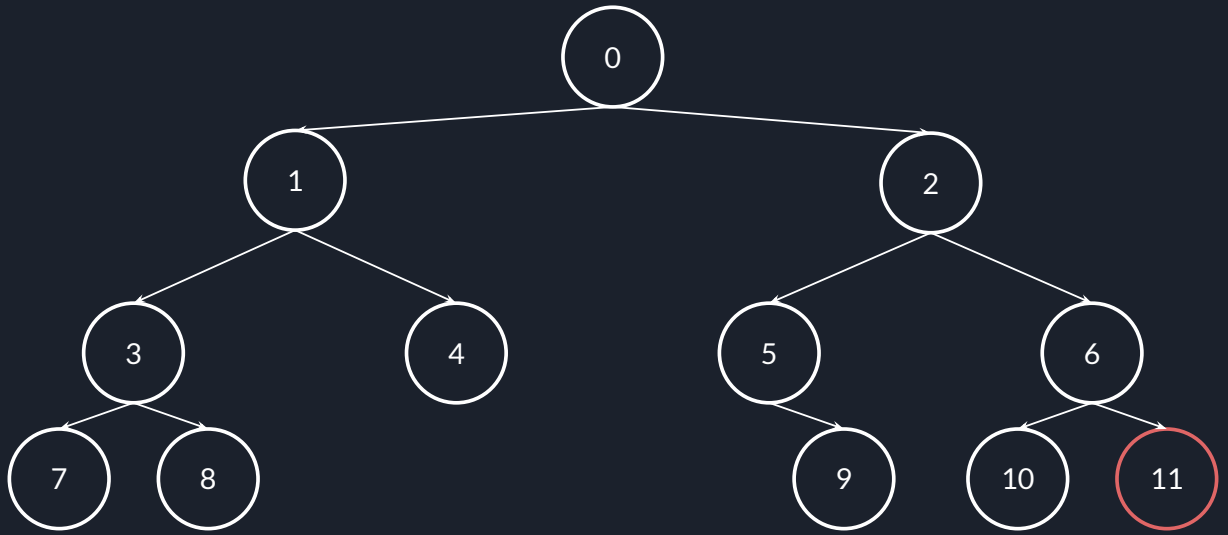
Depth-First Search



Visited: 0, 1, 3, 7, 8, 4, 2, 5, 9, 6, 10

Stack: [11]

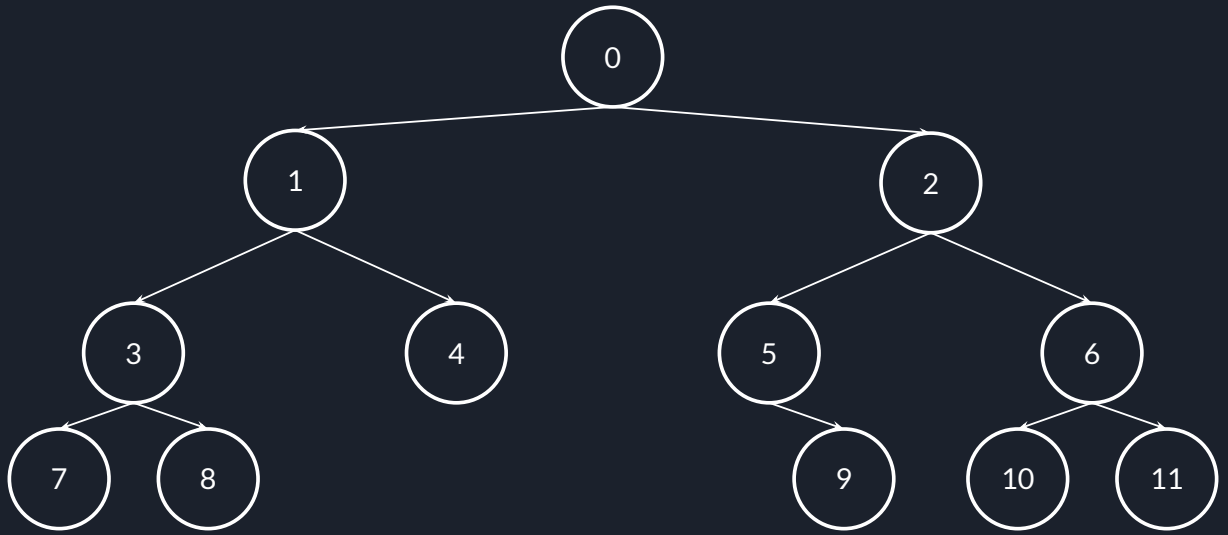
Depth-First Search



Visited: 0, 1, 3, 7, 8, 4, 2, 5, 9, 6, 10, 11

Stack: []

Depth-First Search



Visited: 0, 1, 3, 7, 8, 4, 2, 5, 9, 6, 10, 11

Stack: []

二元樹的DFS順序

二元樹的DFS順序分成三種，差別是執行動作的順序(如 cout)：

```
void preorder(Node *node) {
  do_something();
  preorder(node->left);
  preorder(node->right);
}
```

```
void inorder(Node *node) {
  inorder(node->left);
  do_something();
  inorder(node->right);
}
```

```
void postorder(Node *node) {
  postorder(node->left);
  postorder(node->right);
  do_something();
}
```

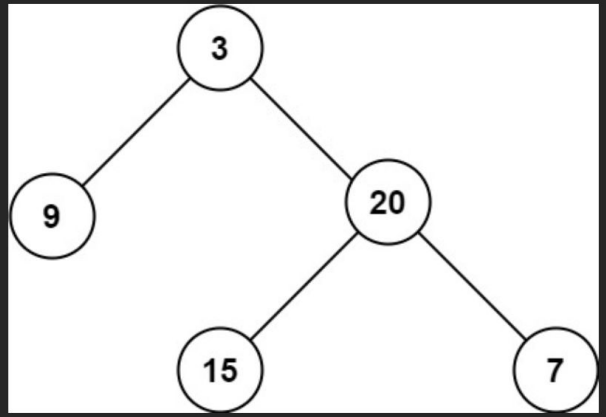
```
preorder:   0 1 3 7 8 4 2 5 9 6 10 11
inorder:    7 3 8 1 4 0 5 9 2 10 6 11
postorder: 7 8 3 4 1 9 5 10 11 6 2 0
```

Leetcode 104 - Maximum Depth of Binary Tree

給予一個二元樹根節點的指標，回傳這棵樹的最大深度

嗯就這樣。

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: 3
```

Example 2:

```
Input: root = [1,null,2]
Output: 2
```

Leetcode 104 - Maximum Depth of Binary Tree

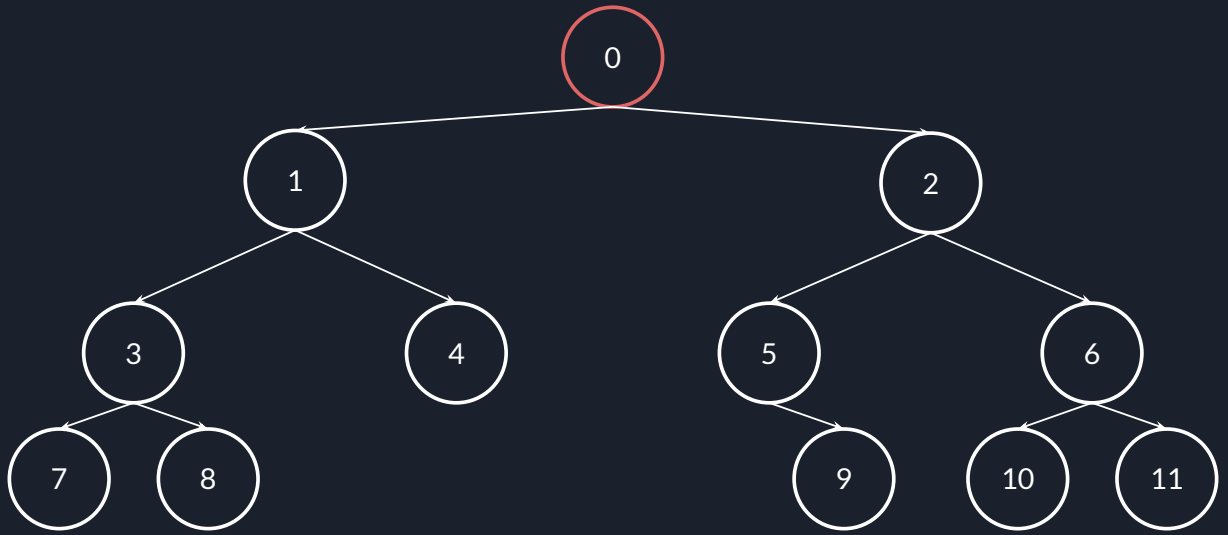
```cpp
class Solution {
public:
  int dfs(TreeNode *node) {
    if (node == nullptr) return 0;

    return max(dfs(node->left), dfs(node->right)) + 1;
  }

  int maxDepth(TreeNode* root) {
    return dfs(root);
  }
};
```

Breadth-First Search

廣度優先搜尋法(breadth-first search, BFS) 是用於遍歷 (traverse) 或搜尋 (search) 的演算法

- 從根出發，會將**同一層遍歷完成**才遍歷下一層
  (相對於深度優先搜尋法DFS)
- 將所在節點的所有子節點放入一個佇列(queue，FIFO 先入先出)

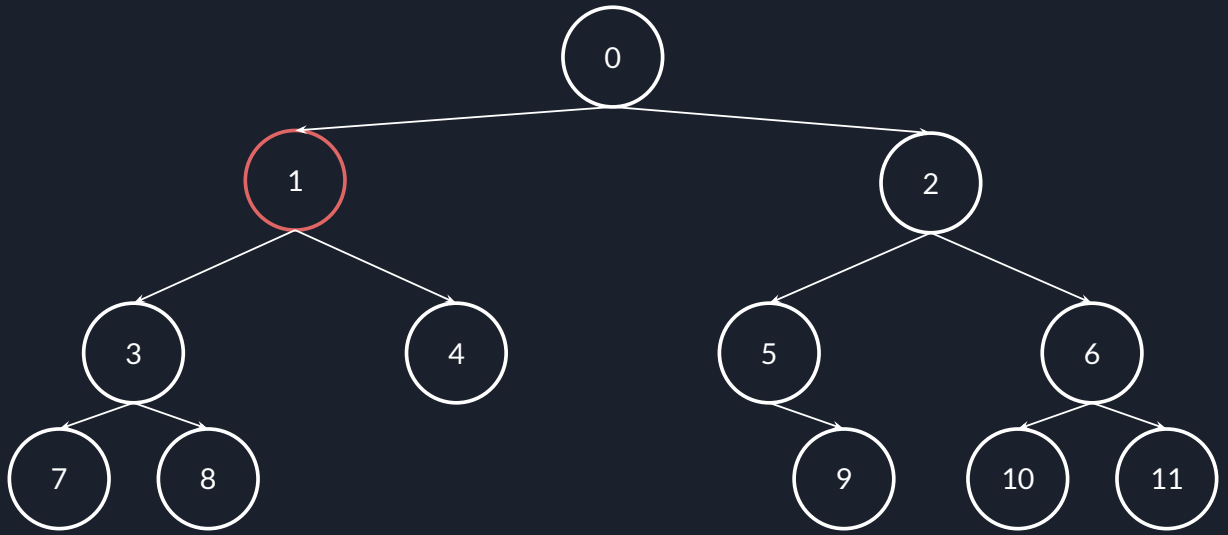
Breadth-First Search



Visited: 0

Queue: [1, 2]

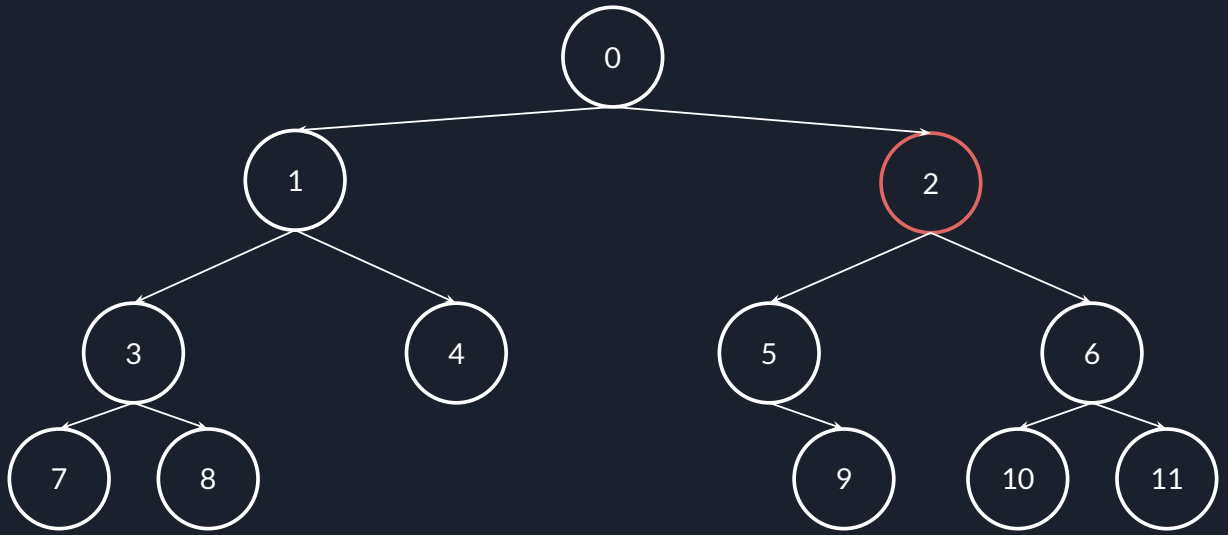
Breadth-First Search



Visited: 0, 1

Queue: [2, 3, 4]

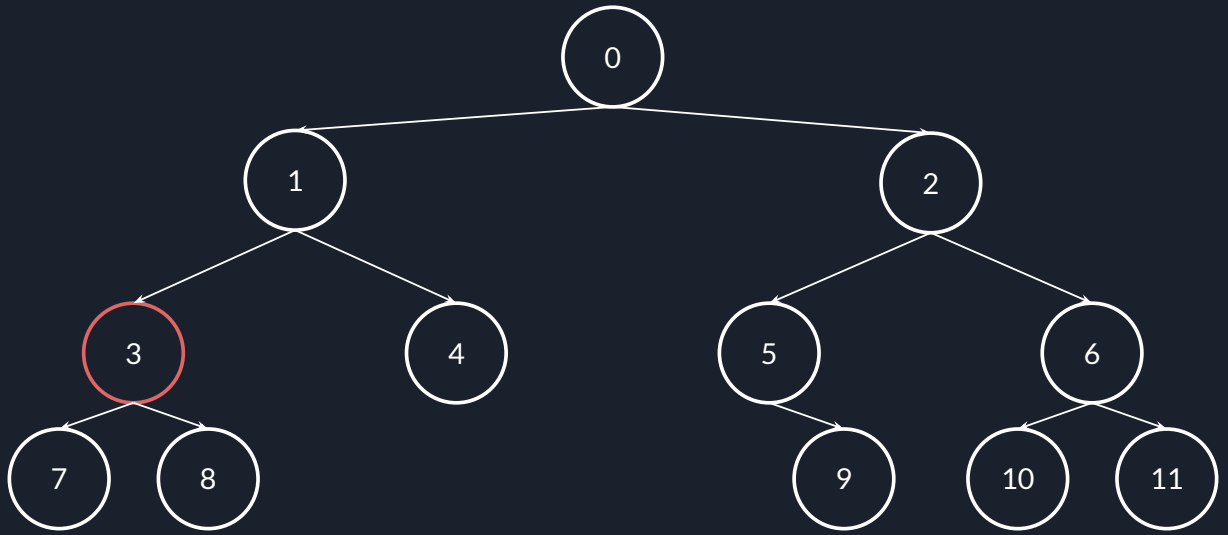
Breadth-First Search



Visited: 0, 1, 2

Queue: [3, 4, 5, 6]

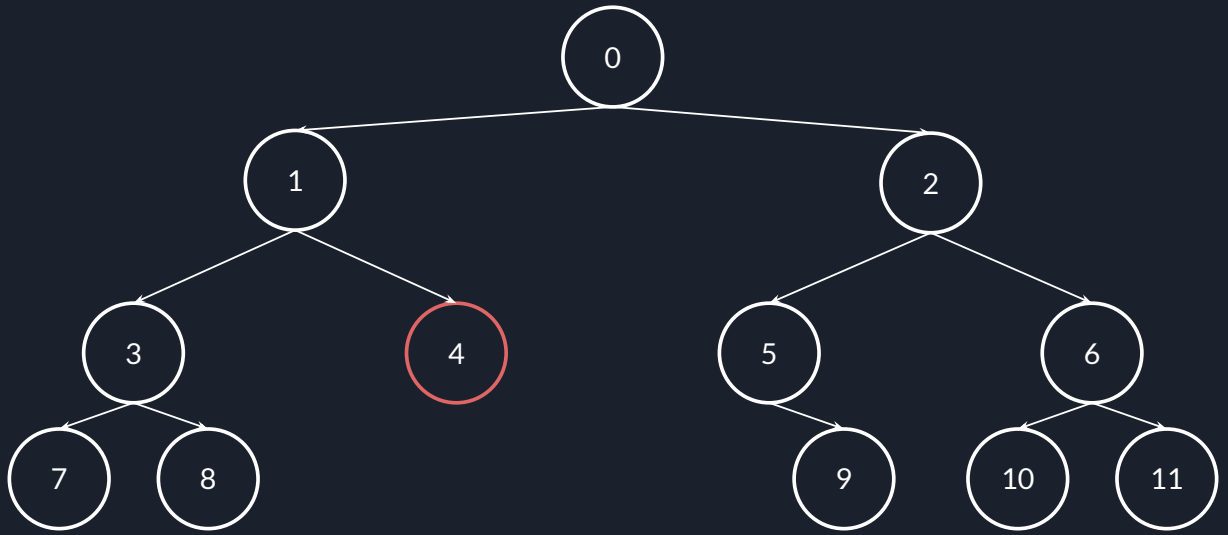
Breadth-First Search



Visited: 0, 1, 2, 3

Queue: [4, 5, 6, 7, 8]

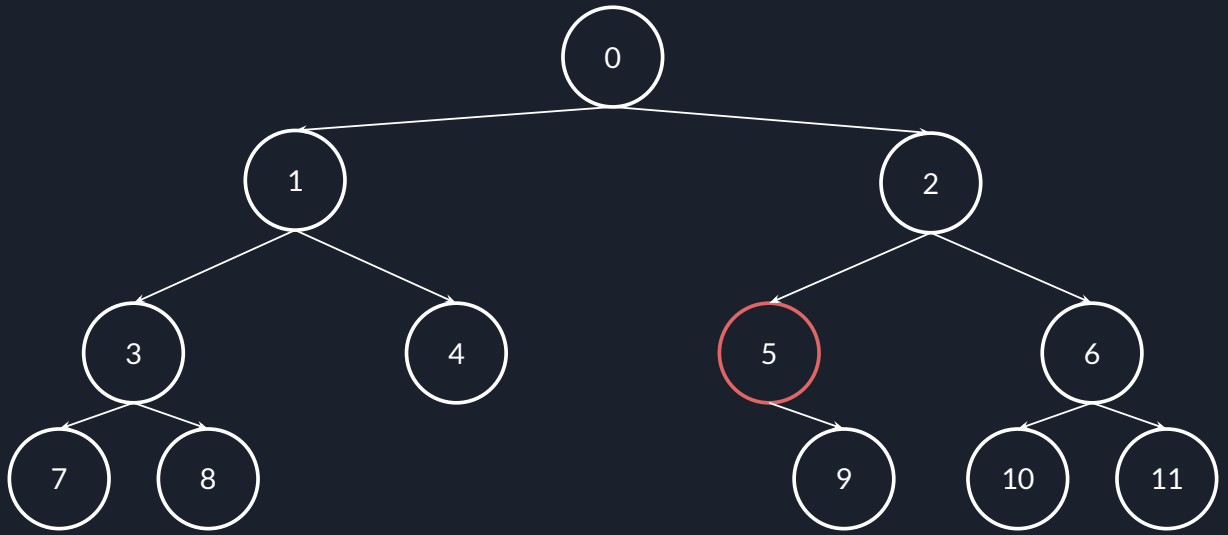
Breadth-First Search



Visited: 0, 1, 2, 3, 4

Queue: [5, 6, 7, 8]

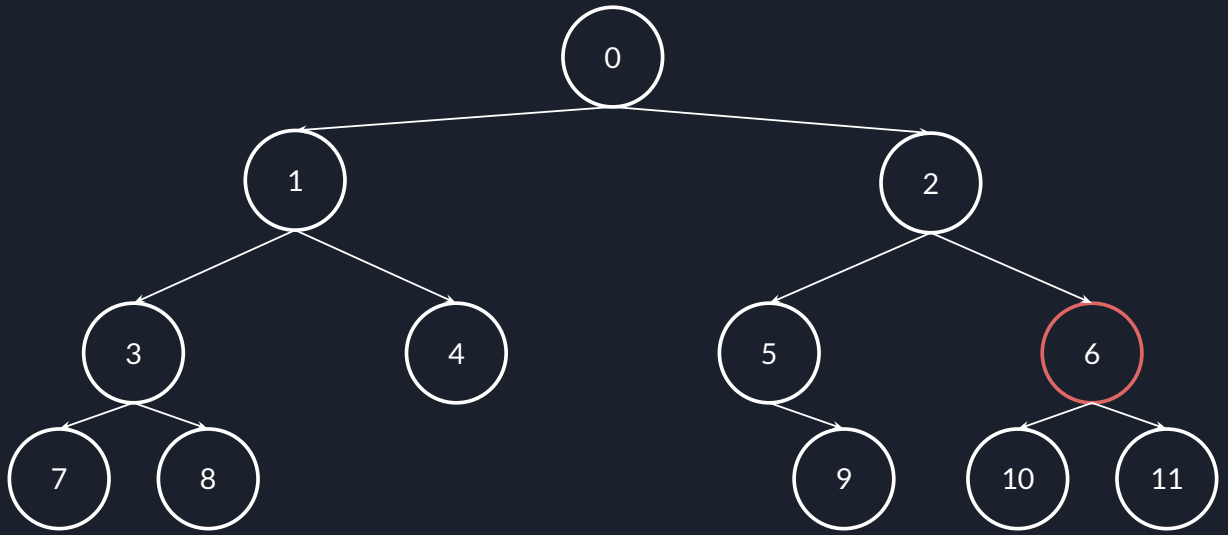
Breadth-First Search



Visited: 0, 1, 2, 3, 4, 5

Queue: [6, 7, 8, 9]

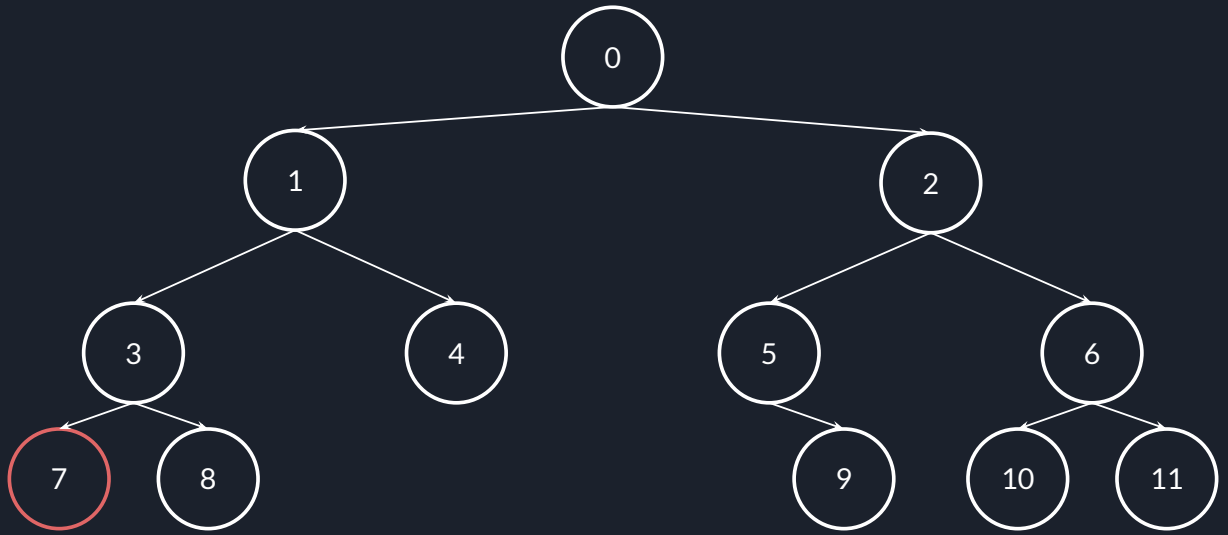
Breadth-First Search



Visited: 0, 1, 2, 3, 4, 5, 6

Queue: [7, 8, 9, 10, 11]

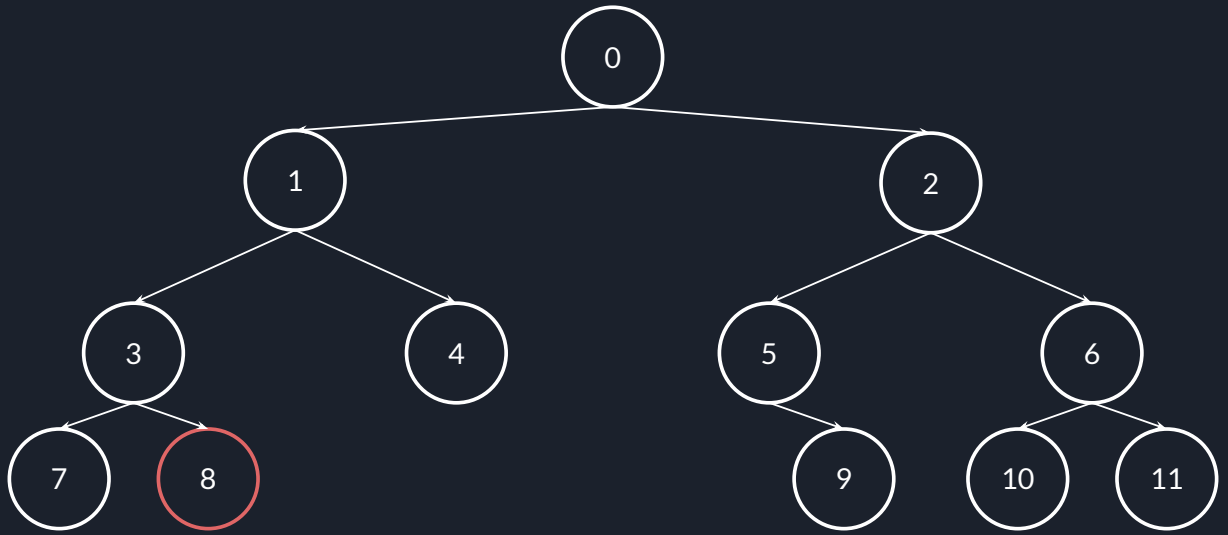
Breadth-First Search



Visited: 0, 1, 2, 3, 4, 5, 6, 7

Queue: [8, 9, 10, 11]

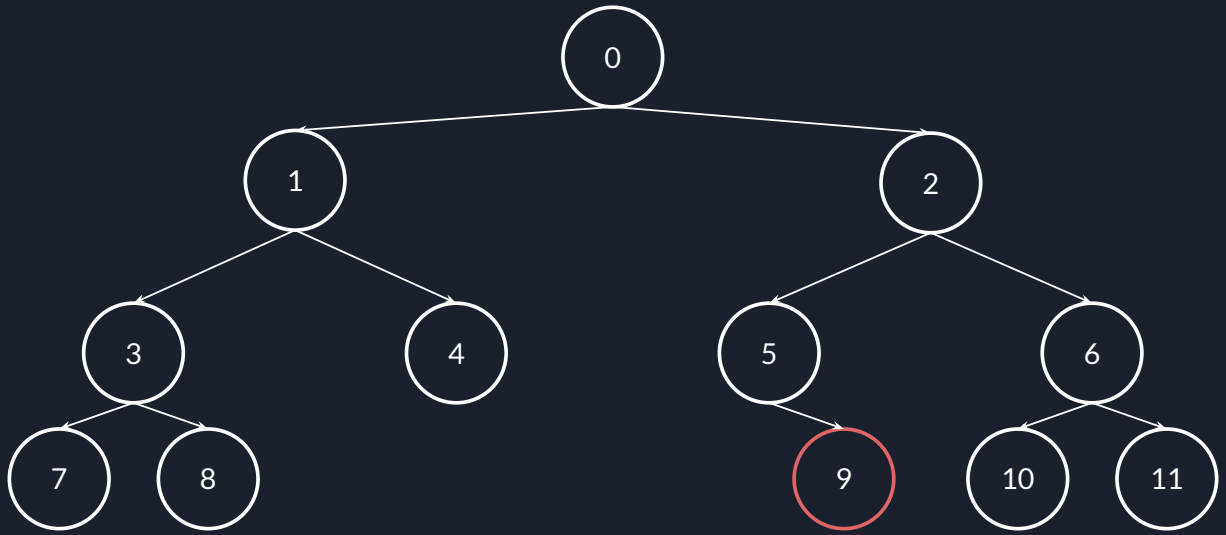
Breadth-First Search



Visited: 0, 1, 2, 3, 4, 5, 6, 7, 8

Queue: [9, 10, 11]

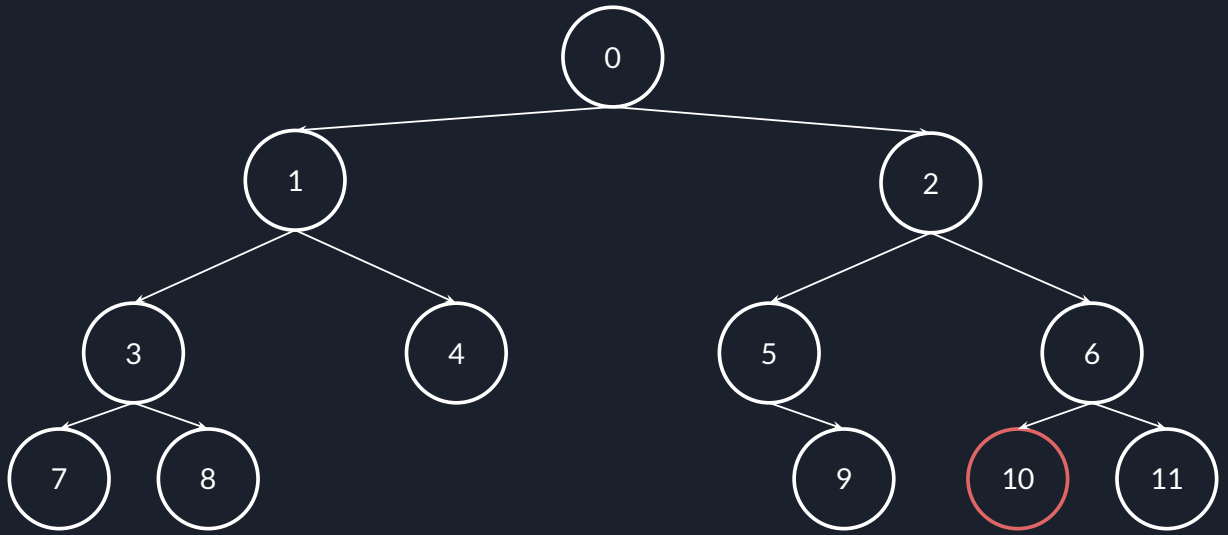
Breadth-First Search



Visited: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Queue: [10, 11]

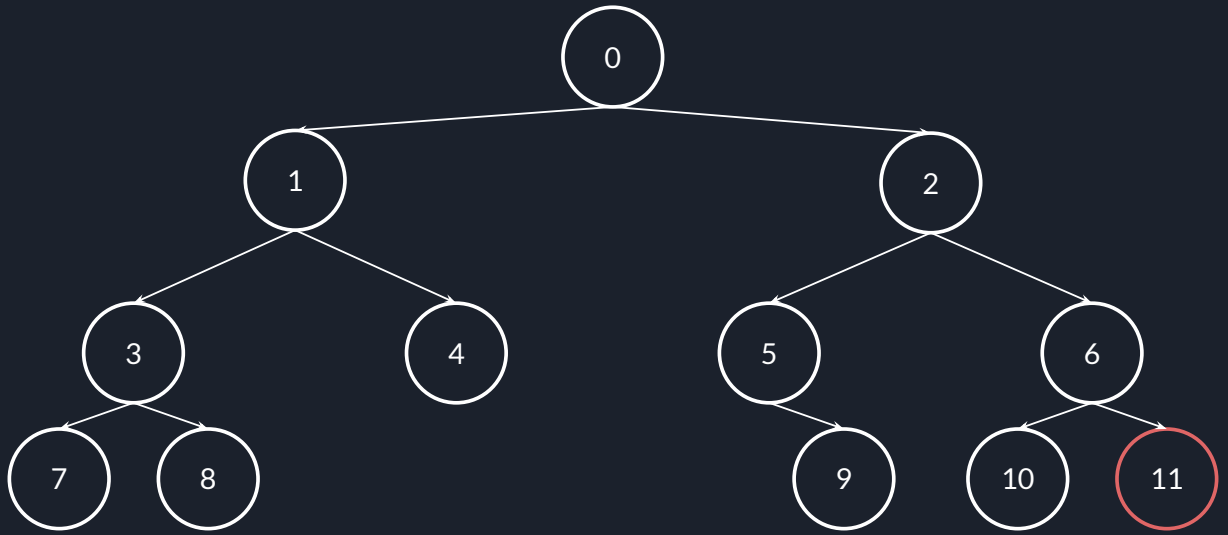
Breadth-First Search



Visited: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Queue: [11]

Breadth-First Search



Visited: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11

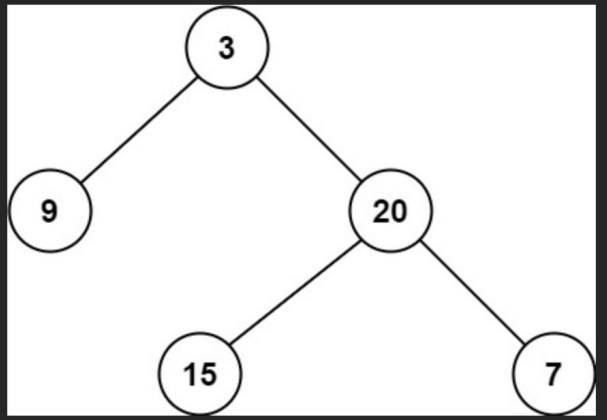
Queue: []

Leetcode 111 - Minimum Depth of Binary Tree

給予一個二元樹根節點的指標，回傳這棵樹任意葉節點的最小深度

嗯。

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: 2
```

Example 2:

```
Input: root = [2,null,3,null,4,null,5,null,6]
Output: 5
```

Leetcode 111 - Minimum Depth of Binary Tree

```cpp
class Solution {
public:
  int minDepth(TreeNode* root) {
    if (root == nullptr) return 0;

    queue<pair<TreeNode *, int>> q;
    q.emplace(root, 0);

    while (!q.empty()) {
      pair<TreeNode *, int> p = q.front();
      q.pop();
      if (p.first == nullptr) continue;
      if (p.first->left == nullptr && p.first->right == nullptr) return p.second + 1;
      q.emplace(p.first->left, p.second + 1);
      q.emplace(p.first->right, p.second + 1);
    }

    return INT_MAX;
  }
};
```